

Aula modelo

Disciplina: Programação aplicada à engenharia cartográfica

Maurício C. M. de Paulo - D.Sc.

2 de junho de 2026



1 Código

2 Exercícios

3 Exercícios com EDGV



1 Código

2 Exercícios

3 Exercícios com EDGV



chaves relacionamentos n-m tipos de dados

https://www.w3schools.com/SQL/sql_quickref.asp distinct dates



1 Código

2 Exercícios

3 Exercícios com EDGV

Criação da tabela

```
CREATE TABLE cidades (  
    id SERIAL PRIMARY KEY,  
    nome TEXT,  
    populacao INTEGER,  
    longitude REAL,  
    latitude REAL  
);
```

Carga dos dados

```
INSERT INTO cidades  
(nome, populacao, longitude, latitude)  
VALUES  
('Rio de Janeiro', 6748000, -43.1729, -22.9068),  
('São Paulo', 12330000, -46.6333, -23.5505),  
('Belo Horizonte', 2531000, -43.9386, -19.9208),  
('Campinas', 1214000, -47.0608, -22.9056);
```



Enunciado

A tabela possui colunas de longitude e latitude. Crie uma coluna espacial geom e preencha-a com geometrias do tipo ponto no sistema WGS84 (EPSG:4326).

1. Adicionar a coluna geom

```
ALTER TABLE cidades
ADD COLUMN geom geometry(Point, 4326);
```

2. Preencher a geometria a partir das coordenadas

```
UPDATE cidades
SET geom = ST_SetSRID(
    ST_MakePoint(
        longitude,
        latitude
    ),
    4326
);
```



Enunciado

Carregue o arquivo br.json na tabela

Enunciado

Carregue o arquivo br.json na tabela

```
import geopandas as gpd
from sqlalchemy import create_engine

# Ler o GeoJSON
gdf = gpd.read_file("br.json")

# Conexão com o PostGIS
engine = create_engine(
    "postgresql+psycopg://postgres:postgres@localhost/geodb"
)

# Criar/substituir tabela no banco
gdf.to_postgis(
    name="brasil",
    con=engine,
    if_exists="replace",
    index=True
)

print("Camada importada com sucesso.")
```



Enunciado

Liste todas as cidades cadastradas e mostre o campo geometria como texto.



Enunciado

Liste todas as cidades cadastradas e mostre o campo geometria como texto.

```
SELECT ST_AsText(geom), *  
FROM cidades;
```



Enunciado

Liste apenas o nome e a população das cidades.



Enunciado

Liste apenas o nome e a população das cidades.

```
SELECT nome, populacao  
FROM cidades;
```



Enunciado

Liste as cidades com população superior a 3 milhões de habitantes.



Enunciado

Liste as cidades com população superior a 3 milhões de habitantes.

```
SELECT *  
FROM cidades  
WHERE populacao > 3000000;
```



Enunciado

Ordene as cidades pela população, da maior para a menor.



Enunciado

Ordene as cidades pela população, da maior para a menor.

```
SELECT nome, populacao  
FROM cidades  
ORDER BY populacao DESC;
```



Enunciado

Determine quantas cidades estão cadastradas.



Enunciado

Determine quantas cidades estão cadastradas.

```
SELECT COUNT(*)  
FROM cidades;
```



Enunciado

Calcule a população média das cidades.



Enunciado

Calcule a população média das cidades.

```
SELECT AVG(populacao)  
FROM cidades;
```



Enunciado

Determine a menor e a maior população cadastradas.



Enunciado

Determine a menor e a maior população cadastradas.

```
SELECT
    MIN(populacao) AS menor,
    MAX(populacao) AS maior
FROM cidades;
```



Enunciado

Liste as cidades cujo nome começa com a letra "C".



Enunciado

Liste as cidades cujo nome começa com a letra "C".

```
SELECT *  
FROM cidades  
WHERE nome LIKE 'C%';
```



Enunciado

Ordene as cidades da mais oeste para a mais leste utilizando a longitude.



Enunciado

Ordene as cidades da mais oeste para a mais leste utilizando a longitude.

```
SELECT nome, longitude
FROM cidades
ORDER BY longitude;
```



Enunciado

Classifique as cidades em Pequena, Média ou Grande e conte quantas existem em cada categoria.

Enunciado

Classifique as cidades em Pequena, Média ou Grande e conte quantas existem em cada categoria.

```
SELECT
    CASE
        WHEN populacao < 2000000
            THEN 'Pequena'
        WHEN populacao < 5000000
            THEN 'Media'
        ELSE 'Grande'
    END AS categoria,
    COUNT(*) AS total
FROM cidades
GROUP BY categoria;
```



1 Código

2 Exercícios

3 Exercícios com EDGV



Objetivo

Importar automaticamente todos os shapefiles contidos em um arquivo ZIP para o banco PostgreSQL/PostGIS.

```
from pathlib import Path
import tempfile
import zipfile

import geopandas as gpd
from sqlalchemy import create_engine

# conexão com o banco
engine = create_engine(
    "postgresql+psycopg2://"
    "postgres:postgres@localhost/geodb"
)

arquivo_zip = "dados.zip"

with tempfile.TemporaryDirectory() as tmpdir:
```



Enunciado

Faça uma consulta que verifique quais linhas de `hid_trecho_drenagem_l` estão dentro de polígonos de `hid_trecho_massa_dagua_a`.

Enunciado

Faça uma consulta que verifique quais linhas de hid_trecho_drenagem_l estão dentro de polígonos de hid_trecho_massa_dagua_a.

```
SELECT
    d.*
FROM hid_trecho_drenagem_l d
JOIN hid_trecho_massa_dagua_a m
ON ST_Within(d.geom, m.geom);
```